Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы трансляции»

**ОТЧЕТ**
к лабораторной работе №3
на тему:
**«СИНТАКСИЧЕСКИЙ АНАЛИЗАТОР»**
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

**СОДЕРЖАНИЕ**

1 Индивидуальное задание ..... 3
2 Описание работы программы ..... 4
3 Ход выполнения программы ..... 5
Вывод ..... 6
Список использованных источников ..... 7
Приложение А (справочное) Исходный код ..... 8

## 1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

На основе результатов лексического анализа разработать синтаксический анализатор для языка Idris, выполняющий разбор исходной программы табличным или нисходящим методом и формирующий дерево разбора. Входные данные считываются из файла input.txt, а при наличии некорректных конструкций должны формироваться сообщения об ошибках.

## 2 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

В ходе лабораторной работы был реализован синтаксический анализатор для подмножества языка Idris на языке C++. В качестве основы был использован разработанный ранее лексический анализатор, который был расширен для поддержки логических строк, переноса строк и более полного набора операторов и разделителей.

Программа читает исходный текст из файла input.txt. После лексического анализа поток токенов преобразуется в логические строки с учётом отступов, что позволяет корректно разбирать конструкции языка Idris, чувствительные к структуре блоков.

Синтаксический анализатор реализован методом рекурсивного спуска. В нём выделены процедуры разбора описаний импортируемых модулей, сигнатур типов, определений функций, выражений, списков, кортежей, условных выражений if-then-else, let-in, case-of и do-блоков. Для представления результата используется древовидная структура CstNode, элементы которой выводятся в иерархическом виде.

В процессе разбора строится синтаксическое дерево, отражающее структуру всей программы. Для каждой функции выделяются узлы имени, параметров и тела. Для составных выражений создаются отдельные узлы операторов, приложений функций, шаблонов сопоставления и ветвей case. Такой формат представления удобен для дальнейших стадий семантического анализа и интерпретации.

Дополнительно программа выводит таблицу токенов, дерево разбора и список обнаруженных ошибок. Если в исходном тексте присутствуют лексические ошибки, такие строки выделяются специальными узлами LexicalErrorLine. Это позволяет сохранить общую структуру разбора корректной части программы и одновременно зафиксировать проблемные места входного файла.

## 3 ХОД ВЫПОЛНЕНИЯ ПРОГРАММЫ

При тестировании использовался файл input.txt, содержащий объявления функций, условные конструкции, списки, кортежи, case-выражения, do-блоки и несколько заведомо ошибочных строк в конце файла. Анализатор успешно построил дерево разбора для корректной части программы и выделил четыре ошибочные строки как отдельные лексические ошибки.

По результатам выполнения программы выводится развёрнутое синтаксическое дерево с узлами Program, Import, TypeSignature, FunctionDefinition, LetIn, If, Case, Do, Apply и другими. Для тестового файла синтаксические ошибки не были обнаружены, так как некорректные конструкции были отфильтрованы ещё на этапе токенизации.

Фрагмент полученного результата подтверждает корректность работы анализатора: в конце отчёта выводятся сообщения о лексических ошибках 1.2.3, 10e, символе # и незакрытой строке, а раздел SYNTAX ERRORS содержит сообщение No syntax errors found.

Основные результаты тестирования: построено полное дерево разбора программы; сохранена информация о структуре функций и выражений; лексические ошибки локализованы по строкам и позициям; корректная часть исходного текста успешно подготовлена к следующему этапу анализа. Все найденные ключевые слова заносятся в таблицу, где каждому из них присваивается уникальный числовой идентификатор. Вывод продемонстрирован ниже, на рисунке 3.1.

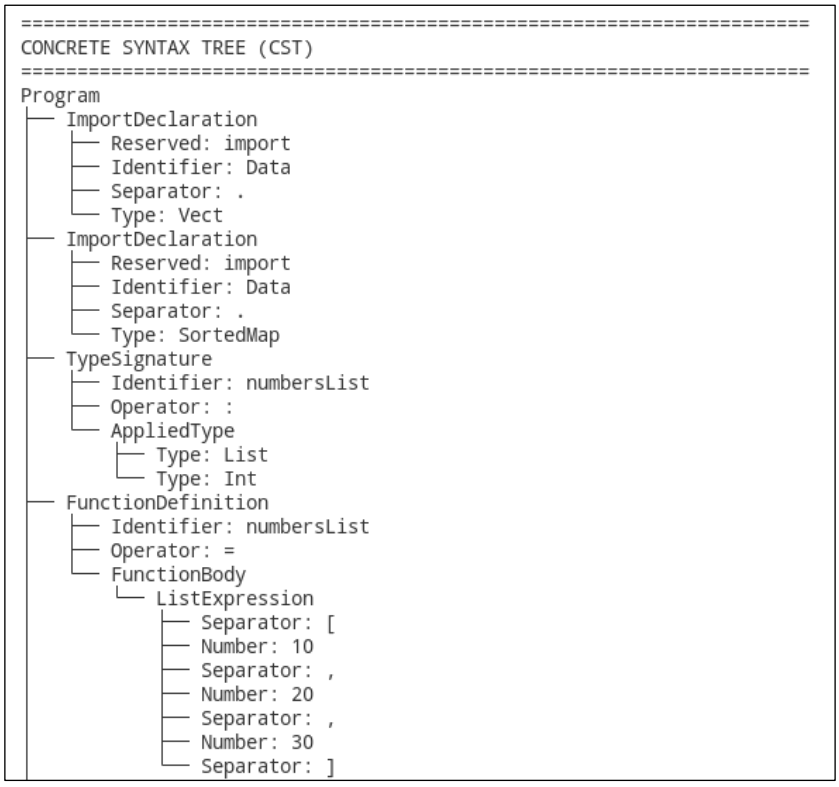



Рисунок 3.1 – Вывод синтаксического дерева

Будем использовать GraphViz для наглядной структуры нашего конкретного абстрактного дерева, результат продемонстрирован на рисунке 3.2.

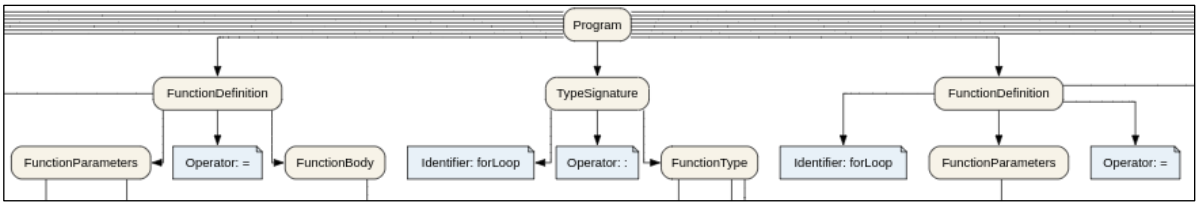



Рисунок 3.2 – Вывод синтаксического дерева в GraphViz

Дерево отображает иерархическую структуру программы: корневой узел Program содержит дочерние узлы Import, TypeSignature, FunctionDefinition и другие. Каждый узел соответствует конкретной языковой конструкции – импорту модуля, сигнатуре типа, определению функции, условному выражению, блоку let-in, case-выражению, do-нотации и т.д. Внутренние узлы, такие как Apply, Operator, Tuple, List, отражают порядок применения операторов и группировку подвыражений. Листовые узлы содержат лексемы – идентификаторы, литералы, операторы. Такое представление наглядно демонстрирует, как рекурсивный спуск преобразует поток токенов в структурированное дерево, пригодное для последующего семантического анализа и интерпретации.

# ВЫВОД

В результате лабораторной работы был разработан синтаксический анализатор для языка Idris, который использует результаты лексического анализа и формирует дерево разбора программы. Реализован разбор основных конструкций языка, используемых во входном файле, обеспечен вывод структуры программы и сообщений об ошибках. Полученный анализатор может использоваться как основа для последующего семантического анализа.

Анализатор успешно обрабатывает логические строки с учётом отступов, что важно для чувствительных к форматированию конструкций Idris (do-блоки, case-выражения, let-in). Программа распознаёт импорты, сигнатуры типов, определения функций, списки, кортежи, условные операторы if-then-else, case-of, do-нотацию и выражения с операторами с учётом приоритетов. Построенное синтаксическое дерево (CST) полностью отражает структуру входной программы и может служить основой для семантического анализа и генерации кода. При наличии лексических ошибок (недопустимые символы, незакрытые строки) анализатор выделяет проблемные строки в отдельные узлы LexicalErrorLine, не нарушая разбор корректной части, и выводит подробные сообщения об ошибках с указанием строки и позиции. Тестирование на предложенном наборе конструкций подтвердило отсутствие синтаксических ошибок в корректной программе и корректную локализацию четырёх лексических ошибок. Таким образом, поставленная задача – разработка синтаксического анализатора на основе результатов лексического анализа, формирующего дерево разбора табличным или нисходящим методом – полностью выполнена. Полученный программный продукт может быть использован в дальнейшем в составе учебного компилятора для языка Idris.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Вирт, Н. Построение компиляторов: ДМК Пресс, 2010. – 192 с.

[2] Ахо, А. В. Компиляторы: принципы, технологии и инструментарий: Вильямс, 2008. – 1184 с.

[3] Документация по Idris [Электронный ресурс]. – Режим доступа: https://docs.idris-lang.org/en/latest. – Дата доступа: 11.02.2026.

# ПРИЛОЖЕНИЕ А
# (справочное)
# Исходный код

Листинг А.1 – Реализация синтаксического анализатора

```
class IdrisParser
{
private:
    vector<Token> tokens;
    vector<LogicalLine> lines;
    size_t lineIndex = 0;
    vector<string> syntaxErrors;

    static string tokenKindToString(TokenKind kind)
    {
        switch (kind)
        {
        case TokenKind::Reserved:
            return "Reserved";
        case TokenKind::Identifier:
            return "Identifier";
        case TokenKind::TypeName:
            return "Type";
        case TokenKind::Number:
            return "Number";
        case TokenKind::StringLiteral:
            return "String";
        case TokenKind::CharLiteral:
            return "Char";
        case TokenKind::Operator:
            return "Operator";
        case TokenKind::Separator:
            return "Separator";
        case TokenKind::Newline:
            return "Newline";
        case TokenKind::EndOfFile:
            return "EOF";
        case TokenKind::Error:
            return "Error";
        }
        return "Unknown";
    }

    static bool isLeafToken(const Token &token)
    {
        return   token.kind   ==   TokenKind::Identifier   ||   token.kind   ==
TokenKind::TypeName || token.kind == TokenKind::Number ||
               token.kind   ==   TokenKind::StringLiteral   ||   token.kind   ==
TokenKind::CharLiteral;
    }

    static string quoteIfNeeded(const Token &token)
    {
        if (token.kind == TokenKind::StringLiteral)
        {
            return "\"" + token.value + "\"";
        }
        if (token.kind == TokenKind::CharLiteral)
        {
            return "'" + token.value + "'";
```

```
        }
        return token.value;
    }

    void recordSyntaxError(const vector<Token> &context, const string &message)
    {
        if (!context.empty())
        {
            const Token &token = context.front();
            syntaxErrors.push_back("Line " + to_string(token.line) + ", column
" + to_string(token.column) + ": " + message);
        }
        else
        {
            syntaxErrors.push_back(message);
        }
    }

    static string tokensToText(const vector<Token> &lineTokens)
    {
        string result;
        for (size_t i = 0; i < lineTokens.size(); ++i)
        {
            if (i != 0U)
            {
                result += ' ';
            }
            result += quoteIfNeeded(lineTokens[i]);
        }
        return result;
    }

    static bool hasLexicalError(const LogicalLine &line)
    {
        for (const auto &token : line.tokens)
        {
            if (token.kind == TokenKind::Error)
            {
                return true;
            }
        }
        return false;
    }

    static vector<LogicalLine> buildLines(const vector<Token> &allTokens)
    {
        vector<LogicalLine> result;
        LogicalLine current;
        bool hasContent = false;

        for (const auto &token : allTokens)
        {
            if (token.kind == TokenKind::Newline)
            {
                if (hasContent)
                {
                    result.push_back(current);
                    current = LogicalLine{};
                    hasContent = false;
                }
                continue;
            }

            if (token.kind == TokenKind::EndOfFile)
```

```
            {
                if (hasContent)
                {
                    result.push_back(current);
                }
                break;
            }

            if (!hasContent)
            {
                current.lineNumber = token.line;
                current.indent = token.column;
                hasContent = true;
            }

            current.tokens.push_back(token);
        }

        return result;
    }

    static bool isFullyWrapped(const vector<Token> &slice, const string
&openToken, const string &closeToken)
    {
        if (slice.size() < 2U || slice.front().value != openToken ||
slice.back().value != closeToken)
        {
            return false;
        }

        int depth = 0;
        for (size_t i = 0; i < slice.size(); ++i)
        {
            if (slice[i].value == openToken)
            {
                depth++;
            }
            else if (slice[i].value == closeToken)
            {
                depth--;
                if (depth == 0 && i + 1 != slice.size())
                {
                    return false;
                }
            }
        }

        return depth == 0;
    }

    static int findTopLevelToken(const vector<Token> &slice, const set<string>
&targets, bool searchFromRight = false)
    {
        int parenDepth = 0;
        int bracketDepth = 0;

        if (!searchFromRight)
        {
            for (size_t i = 0; i < slice.size(); ++i)
            {
                const string &value = slice[i].value;
                if (value == "(")
                {
                    parenDepth++;
```

```
                    continue;
                }
                if (value == ")")
                {
                    parenDepth--;
                    continue;
                }
                if (value == "[")
                {
                    bracketDepth++;
                    continue;
                }
                if (value == "]")
                {
                    bracketDepth--;
                    continue;
                }

                if (parenDepth == 0 && bracketDepth == 0 && targets.count(value)
!= 0U)
                {
                    return static_cast<int>(i);
                }
            }
            return -1;
        }

        for (int i = static_cast<int>(slice.size()) - 1; i >= 0; --i)
        {
            const string &value = slice[static_cast<size_t>(i)].value;
            if (value == ")")
            {
                parenDepth++;
                continue;
            }
            if (value == "(")
            {
                parenDepth--;
                continue;
            }
            if (value == "]")
            {
                bracketDepth++;
                continue;
            }
            if (value == "[")
            {
                bracketDepth--;
                continue;
            }

            if (parenDepth == 0 && bracketDepth == 0 && targets.count(value) !=
0U)
            {
                return i;
            }
        }

        return -1;
    }

    static vector<vector<Token>> splitTopLevel(const vector<Token> &slice,
const string &delimiter)
    {
```

```
        vector<vector<Token>> parts;
        vector<Token> current;
        int parenDepth = 0;
        int bracketDepth = 0;

        for (const auto &token : slice)
        {
            if (token.value == "(")
            {
                parenDepth++;
            }
            else if (token.value == ")")
            {
                parenDepth--;
            }
            else if (token.value == "[")
            {
                bracketDepth++;
            }
            else if (token.value == "]")
            {
                bracketDepth--;
            }

            if (parenDepth == 0 && bracketDepth == 0 && token.value ==
delimiter)
            {
                parts.push_back(current);
                current.clear();
                continue;
            }

            current.push_back(token);
        }

        parts.push_back(current);
        return parts;
    }

    static size_t consumeTerm(const vector<Token> &slice, size_t start)
    {
        if (start >= slice.size())
        {
            return start;
        }

        const string &value = slice[start].value;
        if (value == "(" || value == "[")
        {
            string openToken = value;
            string closeToken = value == "(" ? ")" : "]";
            int depth = 0;

            for (size_t i = start; i < slice.size(); ++i)
            {
                if (slice[i].value == openToken)
                {
                    depth++;
                }
                else if (slice[i].value == closeToken)
                {
                    depth--;
                    if (depth == 0)
                    {
```

```
                        return i + 1;
                    }
                }
            }
            return slice.size();
        }

        return start + 1;
    }

    static vector<vector<Token>> splitApplicationTerms(const vector<Token>
&slice)
    {
        vector<vector<Token>> terms;
        size_t index = 0;

        while (index < slice.size())
        {
            size_t next = consumeTerm(slice, index);
            if (next <= index)
            {
                break;
            }
            terms.emplace_back(slice.begin() + static_cast<long>(index),
slice.begin() + static_cast<long>(next));
            index = next;
        }

        return terms;
    }

    static vector<Token> flattenLines(size_t start, size_t end, const
vector<LogicalLine> &allLines)
    {
        vector<Token> merged;
        for (size_t i = start; i < end; ++i)
        {
            merged.insert(merged.end(), allLines[i].tokens.begin(),
allLines[i].tokens.end());
        }
        return merged;
    }

    size_t findBlockEnd(size_t start, int parentIndent) const
    {
        size_t index = start;
        while (index < lines.size() && lines[index].indent > parentIndent)
        {
            ++index;
        }
        return index;
    }

    shared_ptr<AstNode> parseLeaf(const Token &token) const
    {
        return makeNode(tokenKindToString(token.kind) + ": " +
quoteIfNeeded(token));
    }

    shared_ptr<AstNode> parseTypeTokens(const vector<Token> &slice)
    {
        if (slice.empty())
        {
            return makeNode("Type: <empty>");
```

```
    }

    if (isFullyWrapped(slice, "(", ")"))
    {
        vector<Token> inner(slice.begin() + 1, slice.end() - 1);
        if (inner.empty())
        {
            return makeNode("UnitType");
        }
        auto tupleParts = splitTopLevel(inner, ",");
        if (tupleParts.size() > 1U)
        {
            auto tupleNode = makeNode("TupleType");
            for (const auto &part : tupleParts)
            {
                tupleNode->children.push_back(parseTypeTokens(part));
            }
            return tupleNode;
        }
        return parseTypeTokens(inner);
    }

    int arrowIndex = findTopLevelToken(slice, {"->"});
    if (arrowIndex >= 0)
    {
        auto arrowNode = makeNode("TypeArrow");
        vector<Token> left(slice.begin(), slice.begin() + arrowIndex);
        vector<Token> right(slice.begin() + arrowIndex + 1, slice.end());
        arrowNode->children.push_back(parseTypeTokens(left));
        arrowNode->children.push_back(parseTypeTokens(right));
        return arrowNode;
    }

    auto terms = splitApplicationTerms(slice);
    if (terms.size() > 1U)
    {
        auto applyNode = makeNode("TypeApply");
        for (const auto &term : terms)
        {
            applyNode->children.push_back(parseTypeTokens(term));
        }
        return applyNode;
    }

    if (slice.size() == 1U)
    {
        return parseLeaf(slice.front());
    }

    auto fallback = makeNode("TypeExpr");
    fallback->children.push_back(makeNode(tokensToText(slice)));
    return fallback;
}

shared_ptr<AstNode> parsePatternTokens(const vector<Token> &slice)
{
    if (slice.empty())
    {
        return makeNode("Pattern: <empty>");
    }

    if (isFullyWrapped(slice, "(", ")"))
    {
        vector<Token> inner(slice.begin() + 1, slice.end() - 1);
```

```
            if (inner.empty())
            {
                return makeNode("UnitPattern");
            }
            auto tupleParts = splitTopLevel(inner, ",");
            if (tupleParts.size() > 1U)
            {
                auto tupleNode = makeNode("TuplePattern");
                for (const auto &part : tupleParts)
                {
                    tupleNode->children.push_back(parsePatternTokens(part));
                }
                return tupleNode;
            }
            return parsePatternTokens(inner);
        }

        if (isFullyWrapped(slice, "[", "]"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            auto listNode = makeNode("ListPattern");
            if (!inner.empty())
            {
                auto parts = splitTopLevel(inner, ",");
                for (const auto &part : parts)
                {
                    listNode->children.push_back(parsePatternTokens(part));
                }
            }
            return listNode;
        }

        auto parts = splitApplicationTerms(slice);
        if (parts.size() > 1U)
        {
            auto appNode = makeNode("PatternApply");
            for (const auto &part : parts)
            {
                appNode->children.push_back(parsePatternTokens(part));
            }
            return appNode;
        }

        if (slice.size() == 1U)
        {
            return parseLeaf(slice.front());
        }

        auto fallback = makeNode("Pattern");
        fallback->children.push_back(makeNode(tokensToText(slice)));
        return fallback;
    }

    shared_ptr<AstNode> parseBindingTokens(const vector<Token> &slice, bool
requireIn)
    {
        auto bindingNode = makeNode(requireIn ? "LetBinding" : "LetStatement");
        if (slice.empty())
        {
            bindingNode->children.push_back(makeNode("<empty>"));
            return bindingNode;
        }

        int equalIndex = findTopLevelToken(slice, {"="});
```

```
        if (equalIndex < 0)
        {
            recordSyntaxError(slice, "expected '=' in let binding");
            bindingNode->children.push_back(makeNode(tokensToText(slice)));
            return bindingNode;
        }

        int colonIndex = findTopLevelToken(vector<Token>(slice.begin(),
slice.begin() + equalIndex), {":"});
        if (slice.front().kind == TokenKind::Identifier || slice.front().kind
== TokenKind::TypeName)
        {
            bindingNode->children.push_back(makeNode("Name:        "        +
slice.front().value));
        }
        else
        {
            recordSyntaxError(slice, "let binding must start with an
identifier");
            bindingNode->children.push_back(makeNode("Name: <invalid>"));
        }

        if (colonIndex >= 0)
        {
            vector<Token> typeTokens(slice.begin() + colonIndex + 1,
slice.begin() + equalIndex);
            auto typeNode = makeNode("Type");
            typeNode->children.push_back(parseTypeTokens(typeTokens));
            bindingNode->children.push_back(typeNode);
        }

        int valueStart = colonIndex >= 0 ? equalIndex + 1 : equalIndex + 1;
        vector<Token> valueTokens(slice.begin() + valueStart, slice.end());
        auto valueNode = makeNode("Value");
        valueNode->children.push_back(parseExpressionTokens(valueTokens));
        bindingNode->children.push_back(valueNode);
        return bindingNode;
    }

    shared_ptr<AstNode> parseExpressionTokens(const vector<Token> &slice)
    {
        if (slice.empty())
        {
            return makeNode("Expr: <empty>");
        }

        if (isFullyWrapped(slice, "(", ")"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            if (inner.empty())
            {
                return makeNode("Unit");
            }
            auto tupleParts = splitTopLevel(inner, ",");
            if (tupleParts.size() > 1U)
            {
                auto tupleNode = makeNode("Tuple");
                for (const auto &part : tupleParts)
                {
                    tupleNode-
>children.push_back(parseExpressionTokens(part));
                }
                return tupleNode;
            }
```

```
            return parseExpressionTokens(inner);
        }

        if (slice.front().value == "if")
        {
            int thenIndex = findTopLevelToken(slice, {"then"});
            int elseIndex = findTopLevelToken(slice, {"else"});

            if (thenIndex < 0 || elseIndex < 0 || thenIndex >= elseIndex)
            {
                recordSyntaxError(slice, "invalid if expression");
                return makeNode("If: <invalid>");
            }

            vector<Token>  condTokens(slice.begin()  +  1,  slice.begin()  +
thenIndex);
            vector<Token>   thenTokens(slice.begin()   +   thenIndex   +   1,
slice.begin() + elseIndex);
            vector<Token>   elseTokens(slice.begin()   +   elseIndex   +   1,
slice.end());

            auto ifNode = makeNode("If");
            auto condNode = makeNode("Condition");
            condNode->children.push_back(parseExpressionTokens(condTokens));
            auto thenNode = makeNode("Then");
            thenNode->children.push_back(parseExpressionTokens(thenTokens));
            auto elseNode = makeNode("Else");
            elseNode->children.push_back(parseExpressionTokens(elseTokens));
            ifNode->children = {condNode, thenNode, elseNode};
            return ifNode;
        }

        if (slice.front().value == "let")
        {
            int inIndex = findTopLevelToken(slice, {"in"});
            if (inIndex < 0)
            {
                return  parseBindingTokens(vector<Token>(slice.begin()  +  1,
slice.end()), false);
            }

            vector<Token>  bindingTokens(slice.begin()  +  1,  slice.begin()  +
inIndex);
            vector<Token> bodyTokens(slice.begin() + inIndex + 1, slice.end());

            auto letNode = makeNode("LetIn");
            letNode->children.push_back(parseBindingTokens(bindingTokens,
true));
            auto inNode = makeNode("In");
            inNode->children.push_back(parseExpressionTokens(bodyTokens));
            letNode->children.push_back(inNode);
            return letNode;
        }

        static const vector<set<string>> precedence = {
            {"&&", "||"},
            {">", "<", ">=", "<=", "==", "/="},
            {"++"},
            {"+", "-"},
            {"*", "/"}};

        for (const auto &level : precedence)
        {
            int opIndex = findTopLevelToken(slice, level, true);
```

```
            if (opIndex > 0 && static_cast<size_t>(opIndex + 1) < slice.size())
            {
                auto      opNode      =      makeNode("Operator:      "      +
slice[static_cast<size_t>(opIndex)].value);
                vector<Token> left(slice.begin(), slice.begin() + opIndex);
                vector<Token> right(slice.begin() + opIndex + 1, slice.end());
                opNode->children.push_back(parseExpressionTokens(left));
                opNode->children.push_back(parseExpressionTokens(right));
                return opNode;
            }
        }

        if (isFullyWrapped(slice, "[", "]"))
        {
            vector<Token> inner(slice.begin() + 1, slice.end() - 1);
            auto listNode = makeNode("List");
            if (!inner.empty())
            {
                auto parts = splitTopLevel(inner, ",");
                for (const auto &part : parts)
                {
                    listNode-
>children.push_back(parseExpressionTokens(part));
                }
            }
            return listNode;
        }

        auto parts = splitApplicationTerms(slice);
        if (parts.size() > 1U)
        {
            auto applyNode = makeNode("Apply");
            for (const auto &part : parts)
            {
                applyNode->children.push_back(parseExpressionTokens(part));
            }
            return applyNode;
        }

        if (slice.size() == 1U)
        {
            if (slice.front().kind == TokenKind::Error)
            {
                recordSyntaxError(slice, "lexical error inside expression");
                return makeNode("ErrorExpr");
            }
            return parseLeaf(slice.front());
        }

        recordSyntaxError(slice,   "could   not   parse   expression:   "   +
tokensToText(slice));
        auto fallback = makeNode("Expr");
        fallback->children.push_back(makeNode(tokensToText(slice)));
        return fallback;
    }

    shared_ptr<AstNode> parseCaseBlock(size_t start, size_t end)
    {
        const auto &caseLine = lines[start];
        auto caseNode = makeNode("Case");
        int ofIndex = findTopLevelToken(caseLine.tokens, {"of"});
        if (ofIndex < 0)
        {
```

```
            recordSyntaxError(caseLine.tokens, "case expression must contain
'of'");
            caseNode-
>children.push_back(makeNode(tokensToText(caseLine.tokens)));
            return caseNode;
        }

        vector<Token>      subjectTokens(caseLine.tokens.begin()      +      1,
caseLine.tokens.begin() + ofIndex);
        auto subjectNode = makeNode("Subject");
        subjectNode-
>children.push_back(parseExpressionTokens(subjectTokens));
        caseNode->children.push_back(subjectNode);

        size_t branchStart = start + 1;
        if (branchStart >= end || lines[branchStart].indent <= caseLine.indent)
        {
            recordSyntaxError(caseLine.tokens,  "case  expression  must  have
indented branches");
            return caseNode;
        }

        while (branchStart < end)
        {
            const auto &branchLine = lines[branchStart];
            if (branchLine.indent <= caseLine.indent)
            {
                break;
            }

            size_t branchEnd = branchStart + 1;
            while    (branchEnd    <    end    &&    lines[branchEnd].indent    >
branchLine.indent)
            {
                ++branchEnd;
            }

            auto branchNode = makeNode("Branch");
            int arrowIndex = findTopLevelToken(branchLine.tokens, {"=>"});
            if (arrowIndex < 0)
            {
                recordSyntaxError(branchLine.tokens, "case branch must contain
'=>'");
                branchNode-
>children.push_back(makeNode(tokensToText(branchLine.tokens)));
            }
            else
            {
                vector<Token>         patternTokens(branchLine.tokens.begin(),
branchLine.tokens.begin() + arrowIndex);
                vector<Token> rhsTokens(branchLine.tokens.begin() + arrowIndex
+ 1, branchLine.tokens.end());

                auto patternNode = makeNode("Pattern");
                patternNode-
>children.push_back(parsePatternTokens(patternTokens));
                branchNode->children.push_back(patternNode);

                auto valueNode = makeNode("Value");
                if (!rhsTokens.empty())
                {
                    valueNode-
>children.push_back(parseExpressionTokens(rhsTokens));
                }
```

```
                else if (branchEnd > branchStart + 1)
                {
                    valueNode-
>children.push_back(parseExpressionTokens(flattenLines(branchStart    +    1,
branchEnd, lines)));
                }
                else
                {
                    recordSyntaxError(branchLine.tokens,  "empty  case  branch
body");
                    valueNode->children.push_back(makeNode("<empty>"));
                }
                branchNode->children.push_back(valueNode);
            }

            caseNode->children.push_back(branchNode);
            branchStart = branchEnd;
        }

        return caseNode;
    }

    shared_ptr<AstNode>   parseDoBlock(size_t   start,   size_t   end,   const
vector<Token> &inlineTokens = {})
    {
        auto doNode = makeNode("Do");
        if (!inlineTokens.empty())
        {
            vector<Token> tail(inlineTokens.begin() + 1, inlineTokens.end());
            if (!tail.empty())
            {
                auto statementNode = makeNode("Statement");
                statementNode-
>children.push_back(parseExpressionTokens(tail));
                doNode->children.push_back(statementNode);
            }
        }

        if (start >= end)
        {
            recordSyntaxError(inlineTokens, "do block must contain at least one
statement");
            return doNode;
        }

        size_t statementIndex = start;
        int blockIndent = lines[start].indent;
        while (statementIndex < end)
        {
            if (lines[statementIndex].indent != blockIndent)
            {
                recordSyntaxError(lines[statementIndex].tokens,    "unexpected
indentation inside do block");
                ++statementIndex;
                continue;
            }

            size_t statementEnd = statementIndex + 1;
            while   (statementEnd   <   end   &&   lines[statementEnd].indent   >
lines[statementIndex].indent)
            {
                ++statementEnd;
            }
```

```
            const auto &statementLine = lines[statementIndex];
            auto statementNode = makeNode("Statement");

            if                (!statementLine.tokens.empty()                &&
statementLine.tokens.front().value == "let")
            {
                statementNode-
>children.push_back(parseBindingTokens(vector<Token>(statementLine.tokens.beg
in() + 1, statementLine.tokens.end()), false));
            }
            else         if         (!statementLine.tokens.empty()         &&
statementLine.tokens.front().value == "case")
            {
                statementNode-
>children.push_back(parseCaseBlock(statementIndex, statementEnd));
            }
            else         if         (!statementLine.tokens.empty()         &&
statementLine.tokens.front().value == "do")
            {
                statementNode->children.push_back(parseDoBlock(statementIndex
+ 1, statementEnd, statementLine.tokens));
            }
            else
            {
                vector<Token> expressionTokens = statementLine.tokens;
                if (statementEnd > statementIndex + 1)
                {
                    auto   nested   =   flattenLines(statementIndex   +   1,
statementEnd, lines);
                    expressionTokens.insert(expressionTokens.end(),
nested.begin(), nested.end());
                }
                statementNode-
>children.push_back(parseExpressionTokens(expressionTokens));
            }

            doNode->children.push_back(statementNode);
            statementIndex = statementEnd;
        }

        return doNode;
    }

    shared_ptr<AstNode> parseIndentedExpression(size_t start, size_t end)
    {
        if (start >= end)
        {
            return makeNode("Expr: <empty>");
        }

        const auto &firstLine = lines[start];
        if  (!firstLine.tokens.empty()  &&  firstLine.tokens.front().value  ==
"case")
        {
            return parseCaseBlock(start, end);
        }
        if  (!firstLine.tokens.empty()  &&  firstLine.tokens.front().value  ==
"do")
        {
            return parseDoBlock(start + 1, end, firstLine.tokens);
        }

        return parseExpressionTokens(flattenLines(start, end, lines));
    }
```

```
    shared_ptr<AstNode> parseImport()
    {
        const auto &line = lines[lineIndex];
        auto node = makeNode("Import");
        if (line.tokens.size() < 2U)
        {
            recordSyntaxError(line.tokens, "import statement is incomplete");
            node->children.push_back(makeNode("<missing module name>"));
        }
        else
        {
            string moduleName;
            for (size_t i = 1; i < line.tokens.size(); ++i)
            {
                moduleName += line.tokens[i].value;
            }
            node->children.push_back(makeNode("Module: " + moduleName));
        }
        ++lineIndex;
        return node;
    }

    shared_ptr<AstNode> parseTypeSignature()
    {
        const auto &line = lines[lineIndex];
        auto node = makeNode("TypeSignature");

        int colonIndex = findTopLevelToken(line.tokens, {":"});
        if (colonIndex <= 0)
        {
            recordSyntaxError(line.tokens, "invalid type signature");
            node->children.push_back(makeNode(tokensToText(line.tokens)));
            ++lineIndex;
            return node;
        }

        node->children.push_back(makeNode("Name:              "              +
line.tokens.front().value));
        vector<Token>  typeTokens(line.tokens.begin()  +  colonIndex  +  1,
line.tokens.end());
        auto typeNode = makeNode("Type");
        typeNode->children.push_back(parseTypeTokens(typeTokens));
        node->children.push_back(typeNode);

        ++lineIndex;
        return node;
    }

    shared_ptr<AstNode> parseFunctionDefinition()
    {
        const auto &line = lines[lineIndex];
        auto node = makeNode("FunctionDefinition");

        int equalIndex = findTopLevelToken(line.tokens, {"="});
        if (equalIndex <= 0)
        {
            recordSyntaxError(line.tokens, "function definition must contain
'='");
            node->children.push_back(makeNode(tokensToText(line.tokens)));
            ++lineIndex;
            return node;
        }
```

```
        node->children.push_back(makeNode("Name:                "                +
line.tokens.front().value));

        auto paramsNode = makeNode("Parameters");
        vector<Token> lhsTokens(line.tokens.begin() + 1, line.tokens.begin() +
equalIndex);
        auto parameterSlices = splitApplicationTerms(lhsTokens);
        for (const auto &parameter : parameterSlices)
        {
            paramsNode->children.push_back(parsePatternTokens(parameter));
        }
        node->children.push_back(paramsNode);

        size_t blockEnd = findBlockEnd(lineIndex + 1, line.indent);
        vector<Token>   rhsInline(line.tokens.begin()   +   equalIndex   +   1,
line.tokens.end());

        auto bodyNode = makeNode("Body");
        if (!rhsInline.empty())
        {
            if (rhsInline.front().value == "do")
            {
                bodyNode->children.push_back(parseDoBlock(lineIndex    +    1,
blockEnd, rhsInline));
            }
            else
            {
                vector<Token> fullExpression = rhsInline;
                if (blockEnd > lineIndex + 1)
                {
                    auto nestedTokens = flattenLines(lineIndex + 1, blockEnd,
lines);
                    fullExpression.insert(fullExpression.end(),
nestedTokens.begin(), nestedTokens.end());
                }
                bodyNode-
>children.push_back(parseExpressionTokens(fullExpression));
            }
        }
        else if (blockEnd > lineIndex + 1)
        {
            bodyNode->children.push_back(parseIndentedExpression(lineIndex   +
1, blockEnd));
        }
        else
        {
            recordSyntaxError(line.tokens, "missing function body");
            bodyNode->children.push_back(makeNode("<empty>"));
        }
        node->children.push_back(bodyNode);

        lineIndex = blockEnd;
        return node;
    }

public:
    explicit          IdrisParser(vector<Token>          allTokens)          :
tokens(std::move(allTokens)), lines(buildLines(tokens)) {}

    shared_ptr<AstNode> parseProgram()
    {
        auto root = makeNode("Program");

        while (lineIndex < lines.size())
```

```
        {
            const auto &line = lines[lineIndex];

            if (hasLexicalError(line))
            {
                auto errorNode = makeNode("LexicalErrorLine");
                errorNode->children.push_back(makeNode("Line        "        +
to_string(line.lineNumber) + ": " + tokensToText(line.tokens)));
                root->children.push_back(errorNode);
                ++lineIndex;
                continue;
            }

            if (line.indent != 1)
            {
                recordSyntaxError(line.tokens, "unexpected indentation at top
level");
                auto errorNode = makeNode("SyntaxErrorLine");
                errorNode-
>children.push_back(makeNode(tokensToText(line.tokens)));
                root->children.push_back(errorNode);
                ++lineIndex;
                continue;
            }

            if (!line.tokens.empty() && line.tokens.front().value == "import")
            {
                root->children.push_back(parseImport());
                continue;
            }

            bool hasColon = findTopLevelToken(line.tokens, {":"}) >= 0;
            bool hasEqual = findTopLevelToken(line.tokens, {"="}) >= 0;

            if (hasColon && !hasEqual)
            {
                root->children.push_back(parseTypeSignature());
                continue;
            }

            if (hasEqual)
            {
                root->children.push_back(parseFunctionDefinition());
                continue;
            }

            recordSyntaxError(line.tokens, "unknown top-level construct");
            auto errorNode = makeNode("UnknownTopLevel");
            errorNode-
>children.push_back(makeNode(tokensToText(line.tokens)));
            root->children.push_back(errorNode);
            ++lineIndex;
        }

        return root;
    }

    const vector<string> &getSyntaxErrors() const
    {
        return syntaxErrors;
    }

    const vector<LogicalLine> &getLines() const
    {
```

```
        return lines;
    }
};

string readInputFile()
{
    const vector<string> candidates = {"input.txt", "build/input.txt",
"lab3/build/input.txt"};

    for (const auto &path : candidates)
    {
        ifstream input(path);
        if (!input.is_open())
        {
            continue;
        }

        stringstream buffer;
        buffer << input.rdbuf();
        return buffer.str();
    }

    return {};
}

string tokenKindToString(TokenKind kind)
{
    switch (kind)
    {
    case TokenKind::Reserved:
        return "Reserved";
    case TokenKind::Identifier:
        return "Identifier";
    case TokenKind::TypeName:
        return "Type";
    case TokenKind::Number:
        return "Number";
    case TokenKind::StringLiteral:
        return "String";
    case TokenKind::CharLiteral:
        return "Char";
    case TokenKind::Operator:
        return "Operator";
    case TokenKind::Separator:
        return "Separator";
    case TokenKind::Newline:
        return "Newline";
    case TokenKind::EndOfFile:
        return "EOF";
    case TokenKind::Error:
        return "Error";
    }
    return "Unknown";
}

void printTree(const shared_ptr<AstNode> &node, const string &prefix = "", bool
isLast = true, bool isRoot = true)
{
    if (!node)
    {
        return;
    }

    if (isRoot)
```

```
    {
        cout << node->label << '\n';
    }
    else
    {
        cout << prefix;
        cout << (isLast ? "└── " : "├── ");
        cout << node->label << '\n';
    }

    for (size_t i = 0; i < node->children.size(); ++i)
    {
        bool childIsLast = i + 1 == node->children.size();
        string childPrefix = isRoot ? "" : prefix + (isLast ? "    " : "│   ");
        printTree(node->children[i], childPrefix, childIsLast, false);
    }
}

void printTokens(const vector<Token> &tokens)
{
    cout << string(70, '=') << '\n';
    cout << "TOKEN TABLE\n";
    cout << string(70, '=') << '\n';
    cout << left << setw(14) << "TYPE" << setw(28) << "VALUE" << setw(8) <<
"LINE" << setw(8) << "COLUMN" << '\n';
    cout << string(70, '-') << '\n';

    for (const auto &token : tokens)
    {
        cout << left << setw(14) << tokenKindToString(token.kind) << setw(28)
             << (token.kind == TokenKind::StringLiteral ? "\"" + token.value +
"\"" : token.kind == TokenKind::CharLiteral ? "'" + token.value + "'" :
token.value)
             << setw(8) << token.line << setw(8) << token.column << '\n';
    }
}

void printErrors(const vector<Token> &tokens, const vector<string>
&syntaxErrors)
{
    vector<string> lexicalErrors;
    for (const auto &token : tokens)
    {
        if (token.kind == TokenKind::Error)
        {
            lexicalErrors.push_back("Line " + to_string(token.line) + ", column
" + to_string(token.column) + ": " + token.value);
        }
    }

    cout << "\n" << string(70, '=') << '\n';
    cout << "LEXICAL ERRORS\n";
    cout << string(70, '=') << '\n';
    if (lexicalErrors.empty())
    {
        cout << "No lexical errors found.\n";
    }
    else
    {
        for (const auto &error : lexicalErrors)
        {
            cout << error << '\n';
        }
    }
```

```
    cout << "\n" << string(70, '=') << '\n';
    cout << "SYNTAX ERRORS\n";
    cout << string(70, '=') << '\n';
    if (syntaxErrors.empty())
    {
        cout << "No syntax errors found.\n";
    }
    else
    {
        for (const auto &error : syntaxErrors)
        {
            cout << error << '\n';
        }
    }
}
```